

Herald



Herald — Missing Environment Variables (BLOCKER)

Field	Value
Document	Operator action — provide MTPROTO credentials
Created	2026-05-28
Status	BLOCKING Wave 8 Track B (§11.4.98 full-automation compliance)
Blocks	TestSubscribe_LiveBotAPI rewrite, tests/test_wave6_live_loop.sh rewrite, Wave 6.5 lifecycle scenarios — all currently NON-COMPLIANT per §11.4.98
Authority	HelixConstitution §11.4.98 + Herald §108.m (HelixConstitution commit 6828ff2, Herald commit bbff03c8)
Audit anchor	docs/qa/HRD-LIVE-20260528T082128Z/README.md
Resolution path	Populate 3 required + 1 optional HERALD_MTPROTO_* variables in .env; reply "done"

Table of contents

1. [Why this blocker exists](#)
 2. [What's already in .env](#)
 3. [What you must provide](#)
 4. [Step 1 — Create the Telegram app at my.telegram.org](#)
 5. [Step 2 — Choose which phone / Telegram account to use](#)
 6. [Step 3 — Add the QA account to the chat](#)
 7. [Step 4 — Add the variables to .env](#)
 8. [Step 5 — Verify + reply "done"](#)
 9. [What happens next \(after you reply "done"\)](#)
 10. [Security notes](#)
 11. [Troubleshooting](#)
 12. [Cross-references](#)
-

1. Why this blocker exists

Universal Constitution §11.4.98 Full-Automation Anti-Bluff Mandate (anchored 2026-05-28 by your verbatim instruction):

"Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! Everything MUST BE fully automatic and autonomous! These tests MUST BE able to rerun endless times when needed! [...] No bluff is allowed!"

Herald has **three live tests** that currently require you to hand-send a Telegram message during their execution — making them §11.4 PASS-bluffs at the automation layer (cannot run in CI, cannot validate regressions between manual runs, human dependency masks drift):

Test	Manual action currently required
TestSubscribe_LiveBotAPI	Operator hand-sends a message during a 60s window
tests/ test_wave6_live_loop.sh	Operator hand-sends a message; waits for bot reply
Wave 6.5 lifecycle scenarios	Operator hand-sends each scenario's stimulus

The fix is to drive these tests from a **Telegram user account** (not a bot) via the **MTPROTO protocol** — the same wire protocol Telegram apps use. **Why not a second bot?** Empirically verified 2026-05-28: @pherald_qa_bot (id 8971749017) sent message_id=18 to group -4946584787 (ATMOSphere Development); the worker bot @atmosphere_worker_bot observed **0 updates**. Telegram's privacy boundary is structural: bots cannot see other bots' messages in non-DM contexts. MTPROTO user-impersonation is the only autonomous path.

The harness lives in qaherald/internal/mtpROTO/ (vendored github.com/gotd/tD). Once you provide credentials and complete the one-time login (the §11.4.98(B) permitted exception — configuration, not test driving), **every subsequent test run is fully autonomous**, re-runnable endlessly with no human action.

2. What's already in .env

Audited 2026-05-28 (presence-only, no values echoed):

Variable	Status
HERALD_PROJECT_NAME	✔ Set
HERALD_TGRAM_BOT_TOKEN	✔ Set (@atmosphere_worker_bot, id 8823384001)
HERALD_TGRAM_CHAT_ID	✔ Set (-4946584787, "ATMOSphere Development" group)
HERALD_QA_BOT_TOKEN	✔ Set (@pherald_qa_bot, id 8971749017) — but cannot drive worker bot (bot-to-bot wall, see §1)
HERALD_OPERATOR_IDS	✔ Set
HERALD_CLAUDE_BIN	✔ Set (/Users/milosvasic/.local/bin/claude, v2.1.153)
HERALD_CLAUDE_PROJECT_NAME	✔ Set (Herald)

Variable	Status
HERALD_MTPROTO_APP_ID	✗ MISSING — required
HERALD_MTPROTO_APP_HASH	✗ MISSING — required
HERALD_MTPROTO_PHONE	✗ MISSING — required
HERALD_MTPROTO_PASSWORD	✗ MISSING — required ONLY if 2FA enabled

3. What you must provide

Four variables, one optional. Add to your existing `.env` (it's git-ignored at `.gitignore:28`):

```
# §11.4.98 MTPROTO user-account harness (Wave 8 Track B)
HERALD_MTPROTO_APP_ID=<integer, 5-8 digits, from my.telegram.org/
apps>
HERALD_MTPROTO_APP_HASH=<32-char hex string, from
my.telegram.org/apps>
HERALD_MTPROTO_PHONE=<E.164 phone of QA user account, e.g.
+12025551234>
HERALD_MTPROTO_PASSWORD=<only if 2FA enabled on the account;
otherwise leave blank>
```

Where each value comes from:

Variable	What it is	How to get
HERALD_MTPROTO_APP_ID	App api_id (integer)	https://my.telegram.org/apps — see Step 1 below
HERALD_MTPROTO_APP_HASH	App api_hash (32-char hex)	Same page — see Step 1
HERALD_MTPROTO_PHONE	Phone in E.164 format (+countrycode + number, no spaces / dashes)	YOUR phone (the Telegram user account that will drive QA tests). See Step 2 for account-choice trade-offs.
HERALD_MTPROTO_PASSWORD	Cloud 2FA password	Set in Telegram → Settings → Privacy & Security → Two-Step Verification (only if you've enabled 2FA on the account in HERALD_MTPROTO_PHONE)

4. Step 1 — Create the Telegram app at my.telegram.org

Estimated time: 5 minutes (one-time only).

1. **Open browser** to <https://my.telegram.org/auth>.
2. **Enter the QA-driver phone** in E.164 format (e.g. +12025551234).
 - This must be the same number you'll put in HERALD_MTPROTO_PHONE later.
 - It **MUST** be a Telegram **user account**, not a bot.
3. **Telegram sends a login code** to that phone's Telegram app (in-app notification, not SMS).

4. **Enter the code** at my.telegram.org/auth.
5. **You're now logged in** to Telegram's developer dashboard. Navigate to <https://my.telegram.org/apps>.
6. **First-time:** click **"Create new application"**. Fill in:

Field	Value to enter
App title	Herald QA
Short name	herald_qa (no spaces; alphanumeric + underscore only)
URL	Leave blank, OR https://github.com/vasic-digital/Herald
Platform	Other (from dropdown)
Description	Herald §11.4.98 automation harness – fully-autonomous closed-loop testing of @atmosphere_worker_bot.

7. Click **"Create application"**.
8. **Telegram now displays** two values on the page:
 - App `api_id` — small integer (5-8 digits). → This is your `HERALD_MTPROTO_APP_ID`.
 - App `api_hash` — 32-character lowercase hex string. → This is your `HERALD_MTPROTO_APP_HASH`.
9. **⚠ COPY BOTH VALUES IMMEDIATELY.** Telegram does NOT let you re-display the `api_hash` after you navigate away. If you lose it, you must revoke the app + create a new one (which invalidates any existing sessions).

Common mistakes to avoid:

- Don't use a `https://...telegram.org/...` URL anywhere — Telegram's "URL" field is descriptive only, not a redirect.
- Don't put your **bot token** here — `my.telegram.org/apps` is for user-account apps. Bots use `@BotFather` instead.
- Don't share the `api_hash` with another project — Telegram's terms say one app = one purpose; sharing risks a rate-limit ban on both projects.

5. Step 2 — Choose which phone / Telegram account to use

`HERALD_MTPROTO_PHONE` must be a **real Telegram user account** (not a bot). The account will appear as the "sender" of every test-driver message in your group chat -4946584787. Three options:

Option	Pros	Cons	When to pick
(a) Your personal Telegram account	Fastest setup — no new SIM, no new login. The account already has	Test messages appear "from you" in the QA chat. If a bug deletes group messages, it deletes them from YOUR account's perspective. Production CI	First-cycle proof-of-concept. Get Wave 8 Track B working, then migrate to (b).

Option	Pros	Cons	When to pick
	Telegram installed and configured. Clean separation — purpose-built	runs use your personal session — not best practice long-term.	
(b) Dedicated QA SIM	account, isolated from your personal Telegram. Best long-term posture.	Requires a physical SIM card and ~\$10-20/month.	Production CI, when you've validated the harness works.
(c) Voice-over-IP number (Google Voice / Twilio / TextNow)	No physical SIM; free or cheap.	Telegram increasingly rejects VoIP numbers — works ~60% of the time. The account may be flagged <code>USER_DEACTIVATED</code> even after successful initial signup.	Budget-constrained scenarios; try (c) first, fall back to (b) if Telegram blocks.

Operator recommendation for first cycle: Use **option (a) your personal account**. The session file is portable — you can migrate to (b) later by just changing `HERALD_MTPROTO_PHONE` + the session file path; no test rewrites needed.

6. Step 3 — Add the QA account to the chat

The QA user account (`HERALD_MTPROTO_PHONE`) **MUST** be a **member of** `HERALD_TGRAM_CHAT_ID` (the group -4946584787 — "ATMOSphere Development"). Otherwise its messages won't reach the bot's `getUpdates` poller.

- **If using option (a) — your personal account** and you're already in that group:  **Done. Skip this step.**
 - **If using options (b) or (c) — a new account:**
 1. From an existing member's Telegram (e.g. your personal account):
 - Open the "ATMOSphere Development" group.
 - Tap the group name → **Add Members** → search for the QA account's phone or username → add.
 2. **OR:** generate a group invite link, send it to the QA account, and have it join.
 3. **Verify membership** by logging into Telegram as the QA account and confirming the group is visible in its chat list.
-

7. Step 4 — Add the variables to `.env`

Open `.env` in your normal editor (NOT through chat — never paste secret values into chat). Append to the existing file:

```
# §11.4.98 MTPROTO user-account harness (Wave 8 Track B)
HERALD_MTPROTO_APP_ID=12345678
HERALD_MTPROTO_APP_HASH=0123456789abcdef0123456789abcdef
HERALD_MTPROTO_PHONE=+12025551234
HERALD_MTPROTO_PASSWORD=
```

Substitute the values you collected:

- `HERALD_MTPROTO_APP_ID` — from Step 1.8 (small integer)
- `HERALD_MTPROTO_APP_HASH` — from Step 1.8 (32-char hex)
- `HERALD_MTPROTO_PHONE` — from Step 2 (the QA user account's phone in E.164)
- `HERALD_MTPROTO_PASSWORD` — only if the account in `HERALD_MTPROTO_PHONE` has 2FA enabled; otherwise leave the value blank (Telegram will not prompt for it)

Save the file. The `.env` file is git-ignored — your credentials will NOT be committed.

8. Step 5 — Verify + reply "done"

Optional pre-flight check you can run yourself (validates the `api_id` + `api_hash` without exposing them):

```
set -a; source .env; set +a
echo "APP_ID: $([ -n "$HERALD_MTPROTO_APP_ID" ] && echo PRESENT
    || echo MISSING)"
echo "APP_HASH: $([ ${#HERALD_MTPROTO_APP_HASH} -eq 32 ] && echo
    OK_LENGTH || echo WRONG_LENGTH)"
echo "PHONE: $([ -n "$HERALD_MTPROTO_PHONE" ] && echo PRESENT ||
    echo MISSING)"
echo "PASSWORD: $([ -n "$HERALD_MTPROTO_PASSWORD" ] && echo
    PRESENT_2FA || echo BLANK_NO_2FA)"
```

Expected output:

```
APP_ID: PRESENT
APP_HASH: OK_LENGTH
PHONE: PRESENT
PASSWORD: PRESENT_2FA    # or BLANK_NO_2FA
```

If all four lines are correct, **reply "done"** in chat (a single word; do NOT paste any credential values).

9. What happens next (after you reply "done")

Conductor sequence (fully automated except where noted):

1. **Audit** `.env` — presence checks only; no values echoed; verifies `api_hash` length and phone format.
2. **Vendor** `github.com/gotd/td` as a Git submodule under `submodules/gotd-td/` per Herald's vendoring convention.
3. **Scaffold** `qaherald/internal/mtproto/`:
 - `mtproto.go` — Client interface (Connect, SendMessage, WaitForReply, Close)
 - `session.go` — persistent session file management (`~/ .config/herald/mtproto.session`)
 - `errors.go` — sanitizer wrapping all error paths so `api_hash`, session bytes, and 2FA password text never appear in committed logs (HRD-133 parity)

- Hermetic tests with mocked Telegram MTProto endpoint
- 4. **qa herald mtproto login** — Cobra subcommand for the one-time interactive bootstrap:
 - Connects to Telegram MTProto with your `app_id + app_hash`.
 - Telegram sends a login code to `HERALD_MTPROTO_PHONE`'s Telegram app.
 - CLI prompts: **"Enter code Telegram sent to :"** — you type the 5-6 digit code shown in your Telegram app.
 - If 2FA: prompts **"Enter cloud password:"** — you type your `HERALD_MTPROTO_PASSWORD` (or it's read from env).
 - On success: writes `~/.config/herald/mtproto.session` (chmod 600) + prints "MTProto session active for @<username> (user_id=<id>)".
 - This is the §11.4.98(B) permitted one-time exception — configuration, not test driving. All subsequent runs are fully autonomous.
- 5. **Rewrite the 3 NON-COMPLIANT tests:**
 - `TestSubscribe_LiveBotAPI` → `TestSubscribe_LiveMTProto`
 - `tests/test_wave6_live_loop.sh` → `tests/test_wave6_live_mtproto.sh`
 - Wave 6.5 lifecycle scenarios → MTProto-driven equivalents
- 6. **Verify -count=3 deterministic PASS** per §11.4.98 rule (4) — every test must PASS 3 consecutive automated invocations with self-cleaning state.
- 7. **Capture evidence** under `docs/qa/HRD-LIVE-MTPROTO-<TS>/` with sanitizer audit (token-shape + api_hash-shape regex → 0 matches required).
- 8. **Flip the SKIP-with-reason invariants** in `scripts/e2e_bluff_hunt.sh` to PASS (E17 / E18 / E34 / E63-E70 / E71-E80).
- 9. **Commit + push 4 mirrors** + close the §11.4.98 audit (Task #223) — release-gate item.

Estimated wall-clock time from "done" reply to fully-COMPLIANT closed-loop: ~2-4 hours depending on Telegram code-delivery latency and number of test cycles. The login is ~1-minute interactive; everything else is autonomous.

10. Security notes (composes with §11.4.10)

1. **Never commit** `HERALD_MTPROTO_APP_HASH` **or the session file to git**. Both are sufficient to impersonate the QA account. `.env` and `~/.config/herald/mtproto.session` **MUST** stay out of any tracked path.
 2. **Never share the api_hash with another project**. Each project gets its own `my.telegram.org` app per Telegram's terms; sharing risks rate-limit bans across both.
 3. **Treat the session file like an SSH private key**. `chmod 600`, owned by the running user, never world-readable.
 4. **Rotate the api_hash + invalidate the session** if leaked: `my.telegram.org/apps` → revoke + recreate; then `rm ~/.config/herald/mtproto.session` and re-login.
 5. **HRD-133 sanitizer applies to MTProto errors too** — the harness will `sanitizeMTProtoError()` wrap every error path. Any test transcript or log committed to `docs/qa/` is post-sanitization; the `api_hash`, session bytes, and 2FA password text **NEVER** appear in any committed artefact.
 6. **The session file is bound to your** `HERALD_MTPROTO_PHONE`. Moving it to another machine for the same account is fine (`chmod 600` it on the target). **DO NOT** use the same session file for two simultaneously-running test campaigns — Telegram detects parallel sessions and may invalidate one.
-

11. Troubleshooting

AUTH_KEY_UNREGISTERED on first run: The session file from a previous account is present and stale. Run `rm ~/.config/herald/mtproto.session` and re-run `qaherald mtproto login`.

PHONE_CODE_INVALID: The login code was entered wrong or expired (codes valid ~5 minutes). Re-run; Telegram sends a fresh one.

SESSION_PASSWORD_NEEDED: The QA account has 2FA enabled. Add `HERALD_MTPROTO_PASSWORD=<your cloud password>` to `.env` and re-run. The cloud password is what Telegram asks you for when you reinstall the app — set under Settings → Privacy & Security → Two-Step Verification.

FLOOD_WAIT_<N>: Telegram is rate-limiting login attempts on this phone (often after multiple recent attempts). Wait <N> seconds (sometimes 30s, sometimes hours during aggressive throttling). Avoid hammering re-login attempts.

PHONE_NUMBER_INVALID: Wrong E.164 format. Use `+<country code><number>` with NO spaces or dashes. Examples:

- US: +12025551234
- UK: +447911123456
- Serbia: +381601234567
- Germany: +4915123456789

USER_DEACTIVATED: Telegram has deactivated the QA account (often happens to VoIP / suspicious-pattern accounts). Try option (b) — dedicated SIM. There is no appeal process for deactivated accounts in most cases.

The harness sends messages but @atmosphere_worker_bot's poller doesn't see them: Confirm the QA account is a member of `HERALD_TGRAM_CHAT_ID`. The privacy boundary that blocks bot-to-bot ALSO blocks "user-not-in-chat → bot". Add the QA account to the group (Step 3 above).

api_id is reported invalid even though it's the right value: Make sure you're using the integer (no quotes, no leading zeros). Example correct: `HERALD_MTPROTO_APP_ID=12345678`. Example wrong: `HERALD_MTPROTO_APP_ID="12345678"` (don't quote; `.env` is sourced by shell which strips quotes inconsistently).

You created the app under your personal account but want to drive QA from a different account: The `api_id` + `api_hash` are bound to the `my.telegram.org` account that created them BUT can be used to log in as ANY Telegram user. So this is fine — the `api_id/api_hash` identify the *application*, the phone identifies the *user driving it*. You can re-use one app's credentials to log in many users.

You can't access my.telegram.org because Telegram requires phone confirmation but you're in a country where Telegram is blocked: Use a VPN to a country where Telegram is reachable (e.g. EU / US) for the initial app creation. Once you have the `api_id` + `api_hash`, they work from anywhere (the MTProto client connects to Telegram's IPs which may need their own VPN-around routing — but app creation is the bottleneck).

12. Cross-references

- **HelixConstitution §11.4.98** — <parent>/constitution/Constitution.md, commit 6828ff2 (canonical authority for the full-automation mandate).
 - **Herald §108.m** — docs/guides/HERALD_CONSTITUTION.md r8 (project-binding restatement).
 - **Herald restatements** — CLAUDE.md r14, AGENTS.md r10, QWEN.md (all carry literal anchor 11.4.98).
 - **OPERATOR_CREDENTIALS.md** — docs/guides/OPERATOR_CREDENTIALS.md §"MTProto user-account harness" (the full reference; this blocker doc is the operator-actionable subset).
 - **Live evidence audit** — docs/qa/HRD-LIVE-20260528T082128Z/README.md (the 2026-05-28 honest classification that surfaced the manual-dependency NON-COMPLIANT status).
 - **Empirical bot-to-bot wall test** — docs/qa/HRD-LIVE-20260528T082128Z/04_wave6_closed_loop/pherald_journal/transcript.jsonl + the embedded chat log showing 0-update reception.
 - **Telegram's MTProto docs** — https://core.telegram.org/api/obtaining_api_id (the official Telegram doc for the my.telegram.org app-creation flow).
 - **gotd/td Go library** — <https://github.com/gotd/td> (the MTProto client Herald will vendor).
 - **Operator memory telegram_bot_to_bot_wall** — the prior research note that called the MTProto requirement before this 2026-05-28 cycle empirically confirmed it.
-

Status tracking

Track	Status	Reference
§11.4.98 propagation (HelixConstitution + Herald)	✅ DONE	HelixConstitution 6828ff2, Herald bb f03c8
.env.example expansion + canonical docs guide	✅ DONE	Herald 35fc10c
This blocker document + sibling exports	🕒 in flight	Herald (this commit)
Operator provides MTProto credentials	🕒 AWAITING YOU	Reply "done" when .env is populated
Vendor gotd/td + scaffold qaherald/internal/mtproto/	🕒 blocked-on-creds	Wave 8 Track B (Task #221)
qaherald mtproto login (one-time interactive bootstrap)	🕒 blocked-on-creds	Wave 8 Track B
Rewrite 3 NON-COMPLIANT tests to MTProto-driven	🕒 blocked-on-creds	Wave 8 Track B
§11.4.98 audit of all existing tests	🕒 pending	Task #223

You can open this document and begin Step 1 now. Reply "done" when .env is populated.